

Transformer EncoderLayer – Worked Example

We run one EncoderLayer (Self-Attention $\rightarrow$ FFN), with identity projections for attention ($W_q=W_k=W_v=W_o=I$) so you can see the mechanics. LayerNorm uses $\gamma=1$, $\beta=0$. All tensors are shown a single batch ($B=1$) with sequence length $S=4$.

Config: $D=8$, $H=2$ ($d_k=4$), $d_{ff}=16$, $\text{dropout}=0$

Inputs and PositionalEncoding

Token embeddings BEFORE PositionalEncoding (given):

```
t0 (Alice) = [ 0.2 -0.1  0.3  0.5 -0.4  0.1  0.2 -0.1]
t1 (eats)  = [ 0.   0.2 -0.1  0.1  0.3 -0.2  0.1  0. ]
t2 (two)   = [-0.2  0.1  0.1 -0.1  0.1  0.4 -0.1  0.1]
t3 (apples) = [ 0.1  0.1  0.2 -0.2  0.   0.1  0.3 -0.1]
```

Sinusoidal PositionalEncoding uses frequencies $\omega_i = 10000^{-2i/D}$ for $i=0..D/2-1$.
For $D=8$, $\omega \approx [1. \quad 0.1 \quad 0.01 \quad 0.001]$.

First 4 PE rows (even=sin, odd=cos):

```
PE[0] = [0. 1. 0. 1. 0. 1. 0. 1.]
PE[1] = [0.8415 0.5403 0.0998 0.995 0.01 0.9999 0.001 1. ]
PE[2] = [ 0.9093 -0.4161 0.1987 0.9801 0.02 0.9998 0.002 1. ]
PE[3] = [ 0.1411 -0.99 0.2955 0.9553 0.03 0.9995 0.003 1. ]
```

Embeddings AFTER adding PE ($x = \text{embed} + \text{PE}$):

```
x[0] (Alice) = [ 0.2  0.9  0.3  1.5 -0.4  1.1  0.2  0.9]
x[1] (eats)  = [ 0.8415 0.7403 -0.0002 1.095 0.31 0.8 0.101 1. ]
x[2] (two)   = [ 0.7093 -0.3161 0.2987 0.8801 0.12 1.3998 -0.098 1.1 ]
x[3] (apples) = [ 0.2411 -0.89 0.4955 0.7553 0.03 1.0996 0.303 0.9 ]
```

Multi-Head Self-Attention

Multi-Head Attention (identity projections)

Split heads: $Q, K, V \in [B, H, T, d_k]$ with $B=1, H=2, T=4, d_k=4 \Rightarrow Q.shape=(1, 2, 4, 4)$

Scores = $(Q K^T) / \sqrt{d_k}$, softmax over keys; out = $\text{softmax}(\text{scores}) \cdot V$.

Attention weights per head (rows: query t=0..3, cols: keys k=0..3):

Head 0:

```
[[0.4261 0.2983 0.1629 0.1126]
 [0.3448 0.341  0.1939 0.1203]
 [0.2419 0.2492 0.2673 0.2417]
 [0.1947 0.18   0.2814 0.3439]]
```

Head 1:

```
[[0.262  0.1996 0.2957 0.2428]
 [0.2252 0.2332 0.2996 0.2419]
 [0.2294 0.206  0.3306 0.2341]
 [0.2411 0.2129 0.2996 0.2464]]
```

MHA output (merged heads, $W_o=I$) =

```
[[ 0.478977  0.452613  0.232255  1.194304 -0.000147  1.128648  0.117139
   0.979091]
 [ 0.52247   0.39437   0.2209    1.152089  0.025431  1.119731  0.112545
   0.983244]
 [ 0.505904  0.102587  0.272098  1.053433  0.0188    1.137207  0.105203
   0.986714]
 [ 0.472896 -0.086546  0.312848  0.996576  0.012914  1.125842  0.115009
   0.981216]]
```

Add & Norm → FFN → Add & Norm

After Add & Norm (post-LN) — y1 =

```
[[ -0.486883  0.194449 -0.635281  1.551469 -1.578335  1.080493 -0.852855
   0.726942]
 [ 0.237656 -0.054267 -1.217972  1.36216  -1.07193   0.945273 -1.227125
   1.026206]
 [ 0.18263  -1.261012 -0.468515  0.90841  -0.904983  1.518202 -1.037949
   1.063217]
 [-0.144482 -1.839005 -0.049909  0.895851 -0.817164  1.370442 -0.441185
   1.025453]]
```

Position-wise FFN: Linear(D→d_ff) → ReLU → Linear(d_ff→D) with d_ff=16 (same weights shared across time).

h1.shape=(1, 4, 16), ffn_out.shape=(1, 4, 8)

FFN output =

```
[[ 0.36577  0.71028 -0.963873 -0.719948 -0.786678  0.176332  0.701084
   0.528713]
 [ 0.495684 0.940321 -0.932349 -0.693763 -0.733075  0.35849  0.746741
   0.549046]
 [ 0.430425 0.668293 -0.627335 -0.446801 -0.814162  0.289507  0.634917
   0.529536]
 [ 0.159509 0.71459  0.106537 -0.169086 -0.631464  0.254571  0.826398
   0.012256]]
```

After second Add & Norm (final EncoderLayer output, y2) =

```
[[ -0.096696  0.712579 -1.262706  0.654826 -1.866885  0.990344 -0.120882
   0.989421]
 [ 0.485327  0.60078  -1.694765  0.436228 -1.433701  0.916574 -0.432267
   1.121824]
 [ 0.451816 -0.57607  -1.004975  0.322712 -1.536315  1.470222 -0.414368
   1.286978]
 [-0.148294 -1.320599 -0.105493  0.583971 -1.654163  1.508126  0.232569
   0.903883]]
```

Summary

Why the combo works:

- Self-Attention mixes information ACROSS tokens using content-based weights.
- FFN then nonlinearly recomposes features WITHIN each token.
- Residual + LayerNorm stabilize training and keep shapes [B,S,D] consistent for stacking.